

Movie Ticket Booking System

1. Project Overview / Introduction

The **Movie Ticket Booking System** is an online web application designed to make the movie-going experience simple and seamless for users.

It allows **users** to browse movies, view showtimes, select seats, and book tickets online.

It also provides an **admin panel** for managing movies, theaters, and show schedules.

In short —

- **Users** → Easily browse, choose seats, and book tickets.
 - **Admins** → Manage movies, theaters, and bookings efficiently.
-

2. User Roles & Responsibilities

A. User (Customer)

1. **Authentication**
 - Sign Up / Login / Logout securely using email and password.
 2. **Browse Movies**
 - View a list of movies with filters (genre, language, ratings).
 3. **Check Showtimes**
 - See available theaters and timings for each movie.
 4. **Seat Selection & Booking**
 - Select seats from an interactive seat layout.
 - Confirm booking with price calculation.
 5. **Payment**
 - Pay online using Razorpay / Stripe (or mock payment).
 6. **Booking History**
 - View previous bookings and download e-tickets.
 7. **Profile Management**
 - Edit name, phone number, or password.
 - Cancel booking (if allowed).
-

B. Admin

1. **Dashboard**
 - Overview of total users, movies, bookings, and revenue.
2. **Movie Management**
 - Add / Edit / Delete movie details.
 - Upload posters and trailers.
3. **Theater Management**
 - Add / Edit / Delete theaters.
 - Define screens and seating capacity.
4. **Showtime Management**
 - Assign movies to specific screens and time slots.

5. **Seat Management**
 - Set total seats, seat categories, and price.
 6. **Booking Management**
 - View all bookings and monitor cancellations.
 7. **Reports & Analytics**
 - Daily revenue, most booked movie, total tickets sold.
-

3. Scope of Work (Main Functionalities)

User-Side Functionalities

- **Sign Up / Login / Logout** (JWT-based authentication)
- **Browse Movies** by genre, language, rating.
- **View Movie Details** (trailer, cast, description, rating).
- **Check Showtimes** and available theaters.
- **Seat Selection & Ticket Booking** (dynamic layout).
- **Payment Gateway Integration** (Stripe / Razorpay mock).
- **Booking Confirmation & E-ticket generation.**
- **User Profile & Booking History.**

Admin-Side Functionalities

- **Secure Admin Login.**
 - **Add/Edit/Delete Movies and Theaters.**
 - **Assign Shows** (movie → screen → showtime).
 - **Monitor All Bookings.**
 - **View Reports and Daily Analytics.**
-

4. Database Design / Collections (MongoDB)

Collection	Fields	Description
users	name, email, password, phone, role	Stores user & admin data
movies	moviename, genre, language, duration, cast[], director, releaseDate, description, poster, rating	Movie details
theaters	theatername, location, screens, contact, assignedMovies[]	Theater details
shows	movieId, theaterId, screenNumber, showTime[], date, totalSeats, bookedSeats[], price	Show and seat info
bookings	userId, movieId, theaterId, showId, seats[], totalAmount, status, paymentStatus	Booking records

Collection	Fields	Description
payments	bookingId, userId, amount, paymentMethod, paymentStatus, transactionId	Payment details
reviews	userId, movieId, rating, comment	Movie reviews
notifications	userId, message, type, isRead	User alerts
adminReports	date, totalBookings, totalRevenue, mostBookedMovie	Analytical reports

5. MongoDB Schemas

1. User Schema

Purpose: Stores all information about registered users (both customers and admins).

Fields:

- Name
- Email (unique)
- Password (hashed with bcrypt)
- Phone
- Role → user / admin (default: user)
- CreatedAt → Date of registration

Usage:

- Used for login, signup, profile update, and role-based access control.
-

2. Movie Schema

Purpose: Holds details of each movie shown on the platform.

Fields:

- Movie Name
- Genre (e.g., Action, Drama, Comedy)
- Language (e.g., English, Hindi, Bengali)
- Duration (e.g., "2h 10m")
- Cast → Array of actor names
- Director
- Release Date
- Description (short synopsis)
- Poster (URL of image)
- Rating (average user rating)

Usage:

- Display movies, filter/search, assign to theaters and showtimes.
-

3. Theater Schema

Purpose: Contains information about each theater and its screens.

Fields:

- Theater Name
- Location (city or district)
- Total Screens (number of available screens)
- Contact (phone/email)
- Assigned Movies (list of movies and showtimes)

Usage:

- Manage multiple theaters and connect them with shows.
-

4. Show Schema

Purpose: Defines each movie show based on movie, theater, date, and time.

Fields:

- Movie ID (reference to Movie)
- Theater ID (reference to Theater)
- Screen Number
- Show Time (e.g., "10:30 AM", "2:00 PM")
- Date (specific date for show)
- Total Seats
- Booked Seats (array of booked seat numbers)
- Seat Categories (optional) → e.g., Silver, Gold, Platinum
- Price (ticket price per seat)

Usage:

- Controls available seats and showtime scheduling.
-

5. Booking Schema

Purpose: Stores user's ticket booking details.

Fields:

- User ID (reference to User)
- Movie ID (reference to Movie)
- Theater ID (reference to Theater)

- Show ID (reference to Show)
- Seats (list of booked seat numbers)
- Total Amount
- Status → “Confirmed” / “Cancelled”
- Payment Status → “Paid” / “Unpaid”
- BookedAt (date of booking)

Usage:

- Used to show user booking history and for admin to monitor bookings.
-

6. Payment Schema (Optional but Important)

Purpose: Keeps transaction records of payments.

Fields:

- Booking ID (reference)
- User ID (reference)
- Amount
- Payment Method (Card / UPI / NetBanking)
- Payment Status (Success / Failed / Pending)
- Transaction ID (unique ID from payment gateway)
- Payment Date

Usage:

- Verify payments, handle refunds, and generate reports.
-

7. Review Schema (Optional)

Purpose: Stores user ratings and feedback for movies.

Fields:

- User ID (reference)
- Movie ID (reference)
- Rating (1–5)
- Comment (feedback text)
- CreatedAt (review date)

Usage:

- Display user reviews and calculate average ratings.
-

8. Notification Schema (Optional Future Feature)

Purpose: Sends alerts to users (booking confirmation, offers, etc.).

Fields:

- User ID (reference)
- Message (content)
- Type (booking / offer / reminder)
- IsRead (true/false)
- CreatedAt (timestamp)

Usage:

- For sending updates and promotional alerts.
-

9. Admin Report Schema (Optional)

Purpose: Stores daily summary and performance reports.

Fields:

- Date
- Total Bookings
- Total Revenue
- Most Booked Movie

Usage:

- For admin dashboard analytics and tracking platform performance.
-

6. Seat Booking System

1. How Seat Booking Works

Step 1: Show Creation (Admin)

Admin creates a show with:

- movieId
- theaterId
- screenNumber
- showTime
- date
- totalSeats
- bookedSeats (empty initially)
- price

Step 2: User Selects Movie, Theater & Show

User selects:

- Movie → Theater → Date → Show Time
Then sees the seat layout grid on the frontend.

Step 3: Seat Layout Display

Frontend displays seats like:

A1 A2 A3 A4 A5

B1 B2 B3 B4 B5

C1 C2 C3 C4 C5

Seat colors:

-  Available
-  Selected
-  Booked

Step 4: Seat Selection

User selects seats (e.g., A1, A2).

Frontend calculates price → $\text{total} = \text{seatCount} \times \text{pricePerSeat}$.

Step 5: Backend Validation

When booking request is made:

- Backend checks if seats already exist in `bookedSeats[]`.
- If yes → error (“Seat not available”)
- If no → adds seat numbers into `bookedSeats[]`, creates booking entry, and waits for payment confirmation.

Step 6: Payment & Confirmation

After successful payment:

- Booking status → “confirmed”
- Payment status → “paid”
- `bookedSeats[]` in show remains locked.

Step 7: Booking Cancellation (Optional)

If user cancels:

- Booking status → “cancelled”
- Seats removed from `bookedSeats[]` in show.

2. Seat Booking Features

-  Real-time seat locking
-  Prevents double booking
-  Dynamic pricing possible
-  Works for multiple theaters/screens
-  Supports cancellation & refund

6. Technology Stack

Layer	Technology Used
Frontend	Next.js + MUI or Tailwind CSS
Backend	Node.js + Express.js
Database	MongoDB (with Mongoose ORM)
Authentication	JWT + bcrypt
Payment Gateway	Stripe / Razorpay (mock integration)
Deployment	Render / Vercel / AWS

7. Project Flow

1. **User** visits homepage → sees movies.
 2. Clicks on a movie → checks details & showtimes.
 3. Selects **theater, date, and time**.
 4. Picks **seats** → system calculates total price.
 5. Proceeds to **payment** → on success, booking confirmed.
 6. **Booking details** saved → user sees it in booking history.
 7. **Admin** manages everything from dashboard (movies, theaters, reports).
-

8. Future Enhancements (Optional)

- Email / SMS confirmation
 - Offers & promo codes
 - Push notifications
 - Multi-language support
 - Seat categories (Silver, Gold, VIP)
 - Dynamic pricing
-

9. Deliverables

- Complete MERN project code
- Database schema & seed data
- API documentation (Swagger/Postman)

- Deployment setup guide
- Project report & presentation slides

 Final Schema Summary

No	Schema Name	Description	Required
1	User	Stores user and admin info	
2	Movie	Stores movie details	
3	Theater	Theater & screen info	
4	Show	Showtime & seat layout info	
5	Booking	Booking details	
6	Payment	Payment transactions	Optional
7	Review	User feedback	Optional
8	Notification	Alerts and messages	Optional
9	Admin Report	Daily analytics	Optional